J2EE™ Overview

1

Sang Shin

sang.shin@sun.com
www.plurb.com/j2ee/
Java™ Technology Evangelist
Sun Microsystems, Inc.

2

Disclaimer & Acknowledgments

- Even though Sang Shin is a full-time employee of Sun Microsystems, the contents here are created as his own personal endeavor and thus does not reflect any official stance of Sun Microsystems.
- Sun Microsystems is not responsible for any inaccuracies in the contents.
- Acknowledgments
  - Some slides are borrowed from Mark Hapner's JavaONE 2002 “J2EE Overview”

3

Revision History

- 12/08/2002: draft version with speaker notes (Sang)
- 12/16/2002: speaker notes are cleaned up a bit (Sang)
- 4/9/2003: Speaker notes are revised. (Lucille Wilson)

4

Session Objectives

- Getting a big picture of J2EE architecture and platform
- Understanding the value propositions of J2EE
- Getting high-level exposure of APIs and Technologies that constitute J2EE
- Understanding why J2EE is the platform of choice for development and deployment of both web applications and web services

Agenda

- What is J2EE?
- History of Distributed Computing
- Evolution of Enterprise Application Development Frameworks
- Why J2EE?
- J2EE Platform Architecture
- J2EE APIs and Technologies
- RI, Compatibility Test Suite (CTS)
- BluePrints
- J2EE and Web Services

What is J2EE?

Enterprise Computing

Challenges
- Portability
- Diverse Environments
- Time-to-market
- Core Competence
- Assembly
- Integration

Key Technologies
- J2SE™
- J2EE™
  - JMS
  - Servlet
  - JSP
  - Connector
- XML
  - Data Binding

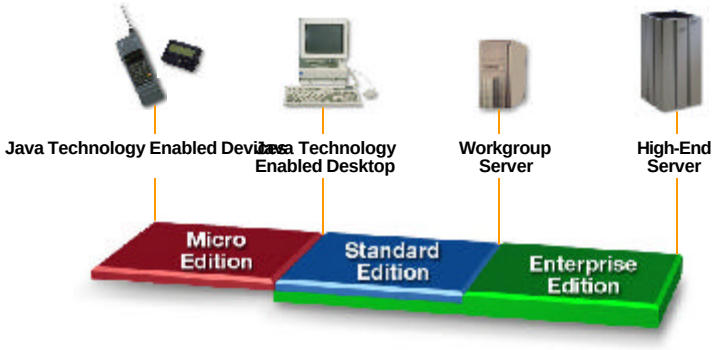
Products
- App Servers
- Web Servers
- Components
- Databases
- Object to DB tools

Legacy Systems
- Databases
- TP Monitors
- EIS Systems

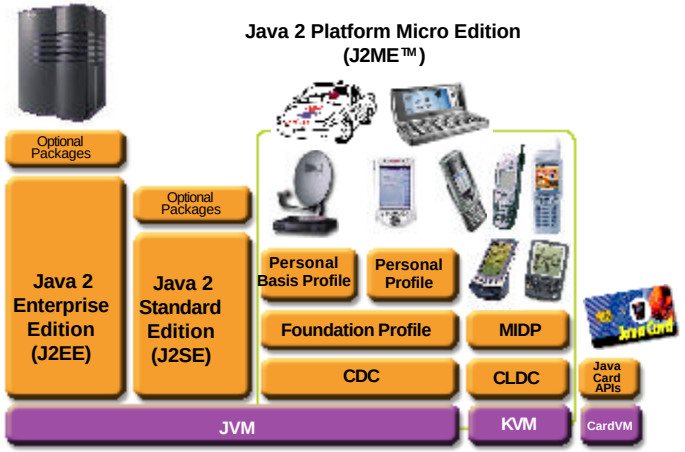
What Is the J2EE?

- Open and standard based platform for
- developing, deploying and managing
- n-tier, Web-enabled, server-centric, and component-based enterprise applications

The Java™ 2 Platform



The Java™ 2 Platform



What Do You Get from J2EE?

- API and Technology specifications
- Development and Deployment Platform
- Reference implementation
- Compatibility Test Suite (CTS)
- J2EE brand
- J2EE Blueprints

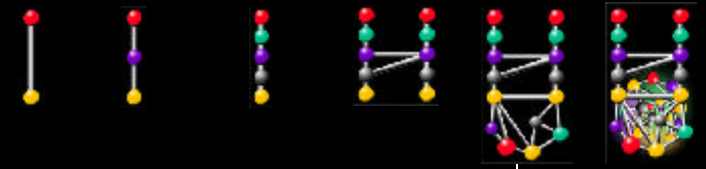
History of Distributed Computing

Platform Evolution

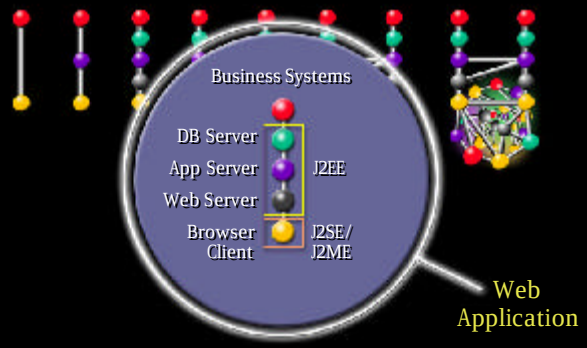
Catch Phrase	The Network Is the Computer	Objects	Legacy to the Web	The Computer Is the Network	Network of Embedded Things	Network of Things
Scale	100s	1,000s	1,000,000s	10,000,000s	100,000,000s	100,000,000s
When/Peak	1984/1987	1990/1993	1996/1999	2001/2003	1998/2004	2004/2007
Leaf Protocol(s)	X	X	+HTTP (+JVM)	+XML Portal	+RM	Unknown
Directory(s)	NS, NS+	+CDS	+LDAP(*)	+UDDI	+Jini	+?
Session	RPC, XDR	+CORBA	+CORBA, RM	+SOAP, XML	+RM/Jini	+?
Schematic						

Communication Patterns

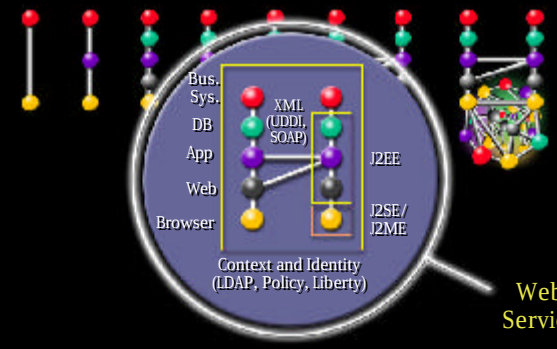
Client-Server | 3-Tier | Web Application | Web Services | Hybrid P2P | Fractal



Communication Patterns: Java™ 2



Communication Patterns: Sun ONE

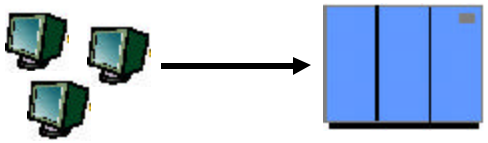


Evolution of Enterprise Application Frameworks

Evolution of Enterprise Application Framework

- Single tier
- Two tier
- Three tier
  - RPC based
  - Remote object based
- Three tier (HTML browser and Web server)
- Proprietary application server
- Standard application server

Single Tier (Mainframe-based)

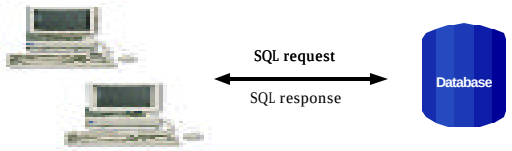


- Centralized model (as opposed distributed model)
- Dumb terminals are directly connected to mainframe
- Presentation, business logic, and data access are intertwined in one monolithic mainframe application

Single-Tier: Pros & Cons

- Pros:
  - No client side management is required
  - Data consistency is easy to achieve
- Cons:
  - Functionality (presentation, data model, business logic) intertwined, difficult for updates and maintenance and code reuse

Two-Tier



- Fat clients talking to back end database
  - SQL queries sent, raw data returned
- Presentation,Business logic and Data Model processing logic in client application

Two-Tier

- Pro:
  - DB product independence (compared to single-tier model)
- Cons:
  - Presentation, data model, business logic are intertwined (at client side), difficult for updates and maintenance
  - Data Model is “tightly coupled” to every client: If DB Schema changes, all clients break
  - Updates have to be deployed to all clients making System maintenance nightmare
  - DB connection for every client, thus difficult to scale
  - Raw data transferred to client for processing causes high network traffic

Three-Tier (RPC based)

- Thinner client: business & data model separated from presentation
  - Business logic and data access logic reside in middle tier server while client handles presentation
- Middle tier server is now required to handle system services
  - Concurrency control, threading, transaction, security, persistence, multiplexing, performance, etc.

Three-tier (RPC based): Pros & Cons

- Pro:
  - Business logic can change more flexibly than 2-tier model
    - Most business logic reside in the middle-tier server
- Cons:
  - Complexity is introduced in the middle-tier server
  - Client and middle-tier server is more tightly-coupled (than the three-tier object based model)
  - Code is not really reusable (compared to object model based)

25

Three-Tier (Remote Object based)

- Business logic and data model captured in objects
  - Business logic and data model are now described in "abstraction" (interface language)
- Object models used: CORBA, RMI, DCOM
  - Interface language in CORBA is IDL
  - Interface language in RMI is Java interface

26

Three-tier (Remote Object based): Pros & Cons

- Pro:
  - More loosely coupled than RPC model
  - Code could be more reusable
- Cons:
  - Complexity in the middle-tier still need to be addressed

27

Three-Tier (Web Server)

- Browser handles presentation logic
- Browser talks Web server via HTTP protocol
- Business logic and data model are handled by "dynamic contents generation" technologies (CGI, Servlet/JSP, ASP)

28

Three-tier (Web Server based): Pros & Cons

- Pro:
  - Ubiquitous client types
  - Zero client management
  - Support various client devices
    - J2ME-enabled cell-phones
- Cons:
  - Complexity in the middle-tier still need to be addressed

Trends

- Moving from single-tier or two-tier to multi-tier architecture
- Moving from monolithic model to object-based application model
- Moving from application-based client to HTML-based client

Single-tier vs. Multi-tier

Single tier	Multi-tier
• No separation among presentation, business logic, database • Hard to maintain	• Separation among presentation, business logic, database • More flexible to change, i.e. presentation can change without affecting other tiers

Monolithic vs. Object-based

Monolithic	Object-based
• 1 Binary file • Recompiled, relinked, redeployed every time there is a change	• Pluggable parts • Reusable • Enables better design • Easier update • Implementation can be separated from interface • Only interface is published

Outstanding Issues & Solution

- Comlexity at the middle tier server still remains
- Duplicate system services still need to be provided for the majority of enterprise applications
  - Concurrency control, Transactions
  - Load-balancing, Security
  - Resource management, Connection pooling
- How to solve this problem?
  - Commonly shared container that handles the above system services
  - Proprietary versus Open-standard based

Proprietary Solution

- Use "component and container" model in which container provides system services in a well-defined but with proprietary manner
- Problem of proprietary solution: Vendor lock-in
- Example: Tuxedo, .NET

Open and Standard Solution

- Use "component and container" model in which container provides system services in a well-defined and as industry standard
- J2EE is that standard that also provides portability of code because it is based on Java technology and standard-based Java programming APIs

Why J2EE?

Platform Value to Developers

- Can use any J2EE implementation for development and deployment
  - Use J2EE RI or Sun ONE Platform Edition which are free for development/deployment and then use high-end commercial J2EE products for actual deployment
- Vast amount of J2EE community resources
  - Many J2EE related books, articles, tutorials, quality code you can use, best practice guidelines, design patterns etc.
- Can use off-the-shelf 3rd-party business components

Platform Value to Vendors

- Vendors work together on specifications and then compete in implementations
  - In the areas of Scalability, Performance, Reliability, Availability, Management and development tools, and so on
- Freedom to innovate while maintaining the portability of applications
- Do not have create/maintain their own proprietary APIs

Platform Value to Business Customers

- Application portability
- Many implementation choices are possible based on various requirements
  - Price (free to high-end), scalability (single CPU to clustered model), reliability, performance, tools, and more
  - Best of breed of applications and platforms
- Large developer pool

J2EE APIs & Technologies

J2EE 1.3 APIs and Technologies

	Version
Java 2 SDK, Standard Edition	1.3
RMI/ IIOP	1.0
JDBC™	3.0
Java Messaging Service	1.0.2b
JNDI	1.2.1
Servlet	2.3
JavaServer Pages™	1.2
JavaMail	1.2
JavaBeans™ Activation Framework	1.0.1
Enterprise JavaBeans	2.0
Java Transaction API	1.0.1
Java Transaction Service	1.1
Connector Architecture	1.0
ECPerf™	1.0

J2EE 1.4 Contents

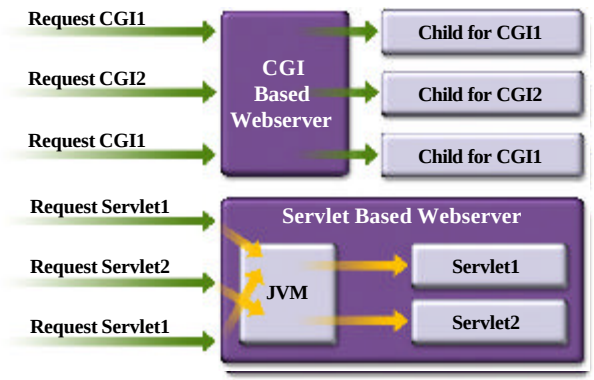
- J2SE 1.4 (improved)
- JAX-RPC (new)
- Web Service for J2EE
- J2EE Management
- J2EE Deployment
- JMX 1.1
- JMS 1.1
- JTA 1.0
- Servlet 2.4
- JSP 2.0
- EJB 2.1
- JAXR
- Connector 1.5
- JACC
- JAXP 1.2
- JavaMail 1.3
- JAF 1.0

Servlet & JSP

What is a Servlet?

- Java™ objects which extend the functionality of a HTTP server
- Dynamic contents generation
- Better alternative to CGI, NSAPI, ISAPI, etc.
  - Efficient
  - Platform and server independent
  - Session management
  - Java-based

Servlet vs. CGI



What is JSP Technology?

- Enables separation of business logic from presentation
  - Presentation is in the form of HTML or XML/XSLT
  - Business logic is implemented as Java Beans or custom tags
  - Better maintainability, reusability
- Extensible via custom tags
- Builds on Servlet technology

EJB (Enterprise Java Beans)

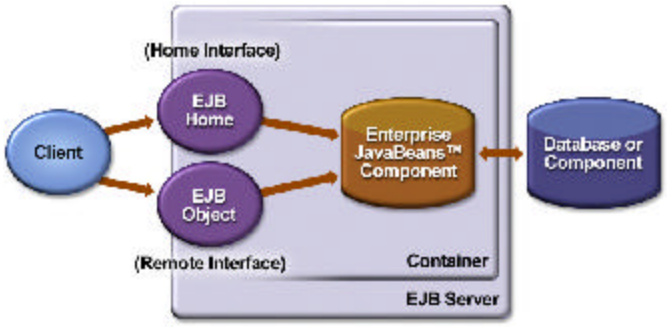
What is EJB Technology?

- Cornerstone of J2EE
- A server-side component technology
- Easy development and deployment of Java technology-based application that are:
  - Transactional, distributed, multi-tier, portable, scalable, secure, ...

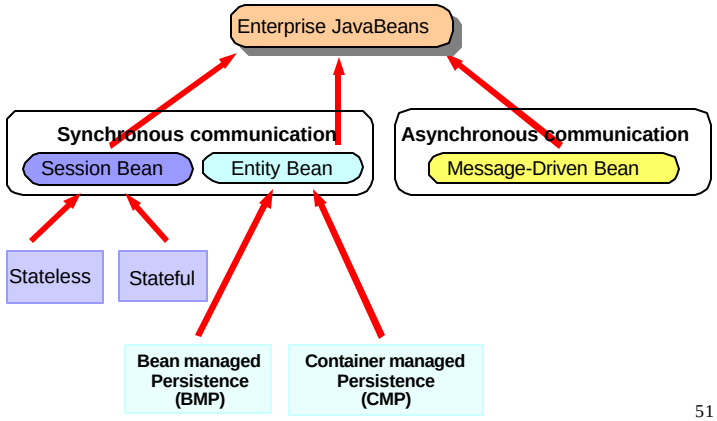
Why EJB Technology?

- Leverages the benefits of component-model on the server side
- Separates business logic from system code
- Provides framework for portable components
  - Over different J2EE-compliant servers
  - Over different operational environments
- Enables deployment-time configuration
  - Deployment descriptor

EJB Architecture



Enterprise JavaBeans



JMS
(Java Message Service)

Java Message Service (JMS)

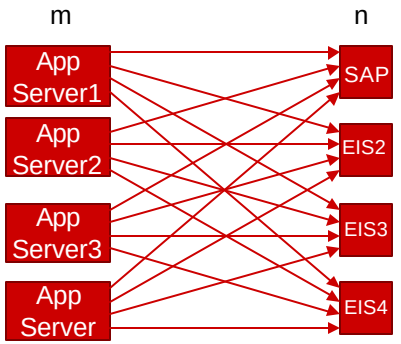
- Messaging systems (MOM) provide
  - Decoupled communication
  - Asynchronous communication
  - Plays a role of centralized post office
- Benefits of Messaging systems
  - Flexible, Reliable, Scalable communication systems
- Point-to-Point, Publish and Subscribe
- JMS defines standard Java APIs to messaging systems

Connector Architecture

Connector Architecture

- Defines standard API for integrating J2EE technology with EIS systems
  - CICS, SAP, PeopleSoft, etc.
- Before Connector architecture, each App server has to provide an proprietary adaptor for each EIS system
  - m (# of App servers) x n (# of EIS's) Adaptors
- With Connector architecture, same adaptor works with all J2EE compliant containers
  - 1 (common to all App servers) x n (# of EIS's) Adaptors

m x n Problem Before Connector Architecture



Connector Architecture

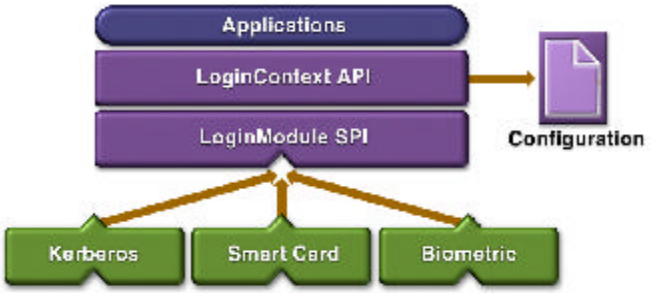
- Defines
  - Connection pooling
  - Security
  - Transaction

JAAS (Part of J2SE 1.4)
(Java Authentication & Authorization Service)

JAAS: Authentication

- Pluggable authentication framework
  - Userid/password
  - Smartcard
  - Kerberos
  - Biometric
- Application portability regardless of authentication schemes underneath
  - JAAS provides authentication scheme independent API
  - Authentication schemes are specified Login configuration file, which will be read by JAAS

JAAS Pluggable Authentication



JAAS: Authorization

- Without JAAS, Java platform security are based on
  - Where the code originated
  - Who signed the code
- The JAAS API augments this with
  - Who's running the code
- User-based authorization is now possible

Other J2EE APIs & Technologies

JNDI

- Java Naming and Directory Interface
- Utilized by J2EE applications to locate resources and objects in portable fashion
  - Applications use symbolic names to find object references to resources via JNDI
  - The symbolic names and object references have to be configured by system administrator when the application is deployed.

JDBC

- Provides standard Java programming API to relational database
  - Uses SQL
- Vendors provide JDBC compliant driver which can be invoked via standard Java programming API

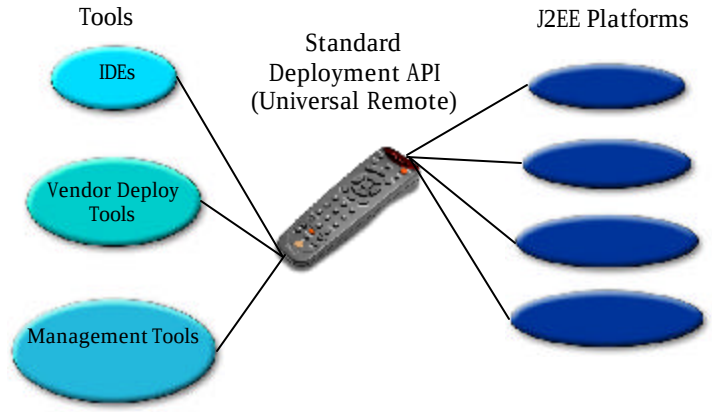
J2EE Management (JSR-77)

- Management applications should be able to discover and interpret the managed data of any J2EE platform
- Single management platform can manage multiple J2EE servers from different vendors
- Management protocol specifications ensure a uniform view by SNMP and WBEM management stations
- Leverages JMX

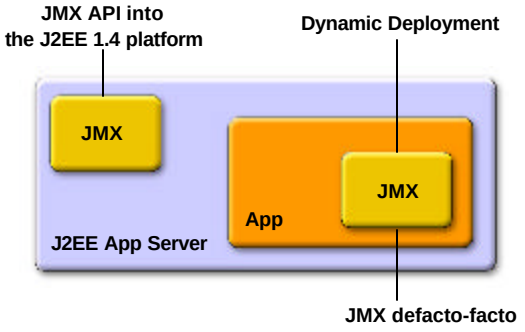
J2EE Management (JSR-77) - J2EE 1.4

- Management information model
  - example: J2EE application server, EJB beans
- Mapping the information model to
  - CIM (Common Information Model)
  - SNMP MIB (Management Information Base)
  - Java APIs for J2EE Management EJB component (MEJB)
    - Exposes managed objects as JMX Managed Beans (Mbeans)

J2EE Deployment (JSR-88) - J2EE 1.4



JMX



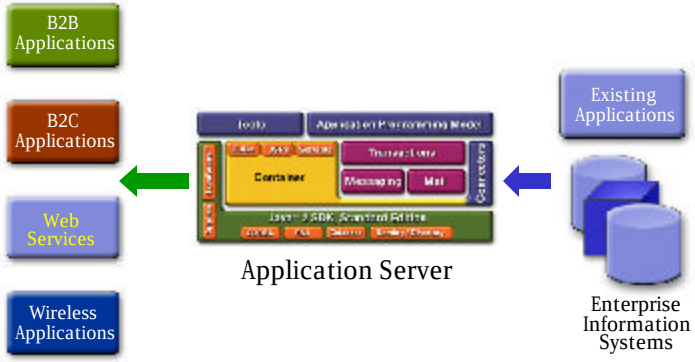
A single technology for the J2EE platform

JACC (Java Authorization Contract for Containers) - J2EE 1.4

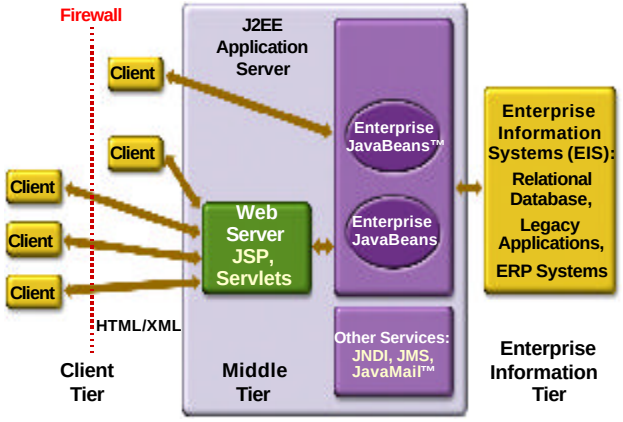
- Defines contract between J2EE containers and authorization policy modules
  - Provider configuration subcontract
  - Policy configuration subcontract
  - Policy enforcement subcontract
- Enable application servers to integrate with enterprise user registries and authorization policy infrastructure

J2EE as End-to-End Architecture

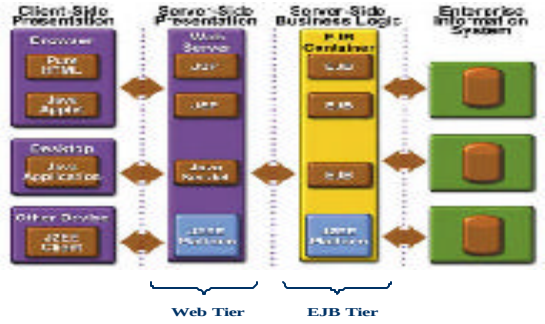
The J2EE Platform Architecture



J2EE is End-to-End Solution

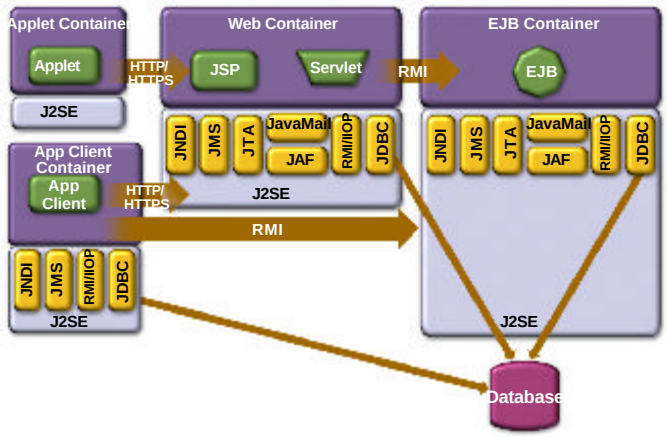


N-tier J2EE Architecture



J2EE Component & Container Architecture

J2EE Containers & Components



Containers and Components

Containers Handle	Components Handle
• Concurrency • Security • Availability • Scalability • Persistence • Transaction • Lifecycle management • Management	• Presentation • Business Logic

Containers & Components

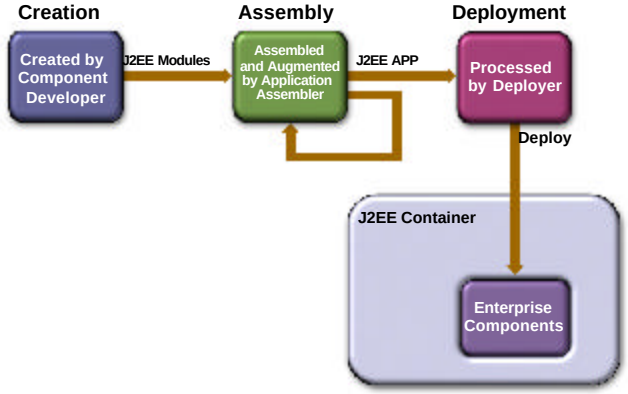
- Containers do their work invisibly
  - No complicated APIs
  - They control by interposition
- Containers implement J2EE
  - Look the same to components
  - Vendors making the containers have great freedom to innovate

J2EE Application Development & Deployment Life Cycle

J2EE Application Development Lifecycle

- Write and compile component code
  - Servlet, JSP, EJB
- Write deployment descriptors for components
- Assemble components into ready-to-deployable package
- Deploy the package on a server

Lifecycle Illustration



J2EE Development Roles

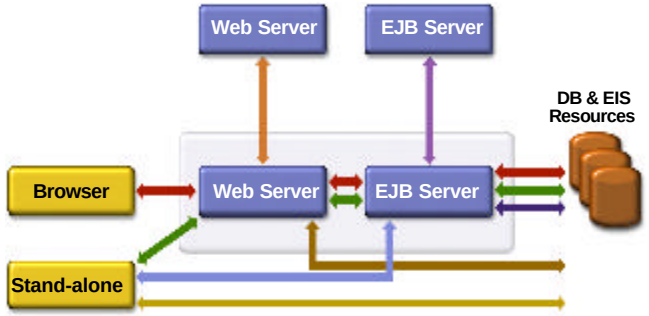
- Component provider
  - Bean provider
- Application assembler
- Deployer
- Platform provider
  - Container provider
- Tools provider
- System administrator

The Deployment Descriptor

- Gives the container instructions on how to manage and control behaviors of the J2EE components
  - Transaction
  - Security
  - Persistence
- Allows declarative customization (as opposed to programming customization)
  - XML file
- Enables portability of code

J2EE Application Anatomies

Possible J2EE Application Anatomies



J2EE Application Anatomies

- 4-tier J2EE applications
  - HTML client, JSP/Servlets, EJB, JDBC/Connector
- 3-tier J2EE applications
  - HTML client, JSP/Servlets, JDBC
- 3-tier J2EE applications
  - EJB standalone applications, EJB, JDBC/Connector
- B2B Enterprise applications
  - J2EE platform to J2EE platform through the exchange of JMS or XML-based messages

Which One to Use?

- Depends on several factors
  - Requirements of applications
  - Availability of EJB tier
  - Availability of developer resource

J2EE Reference Implementation, Compatibility Suite, Brand

What and Why a Reference Implementation?

- Validates specification
- Fully-compliant
- Fully-functional
- Not commercial quality
  - Scalability
  - Performance
- Use it for prototyping
- Java Web Services Developer Pack is production quality, however

Compatibility Test Suite (CTS)

- Ultimate Java™ technology mission:
  - Write Once, Run Anywhere™
  - My Java-based application runs on any compatible Java virtual machines
  - My J2EE based technology-based application will run on any J2EE based Compatible platforms

89

J2EE Application Verification Kit (J2EE AVK)

- How can I test my J2EE application portability?
  - Obtain the J2EE RI 1.3.1 and the J2EE Application Verification Kit (J2EE AVK)
- Self verification of application
  - Static verification
  - Dynamic verification
- Obtain the tests results, verify that all criteria are met

90

Compatible Products for the J2EE Platform (Brand)

ATG
Bea Systems
Borland
Computer Associates
Fujitsu
Hitachi
HP
IBM
IONA
iPlanet
Macromedia
NEC
Oracle
Pramati
SilverStream
Sybase
Talarian
Trifork

Java Compatible™ Enterprise Edition

91

J2EE™ Technology Licensees

AOL	Gemstone	NEC	Sonic
ATG	Hitachi	Nokia	Sybase
BEA Systems	IBM	Merant	Secant
HP Bluestone	Interworld	Oracle Corp.	SpiritSoft
BroadVision	In-Q-My	Cape Clear	Talarian
Borland	IONA	Persistence	TIBCO
Compaq	iPlanet	Pramati	TMAX Soft
CA	Lutris	SAS Institute	TogetherSoft
Fujitsu	Macromedia	Silverstream	Trifork
			WebGain

The J2EE Platform "Ecosystem," Application Servers and...

- Tools
  - IDE's: Borland JBuilder Enterprise, WebGain Visual Cafe', IBM Visual Age for Java™, Forte™ for Java™, Oracle JDeveloper, Macromedia Kawa
  - Modeling, Performance, Testing, etc.
- Enterprise Integration: Connectors, Java Message Service (JMS) API, XML
- Components
- Frameworks
- Applications

Major Investment in Compatibility by the Industry

- Sun has spent scores of engineer years developing tests
- Licensees have spent scores of engineer years passing the tests
- Testing investment on top of specification investment, implementation investment, business investments
- In total, tens of millions of dollars invested in J2EE platform compatibility by the industry

J2EE Blueprint & Pet Store Application

J2EE Blueprint

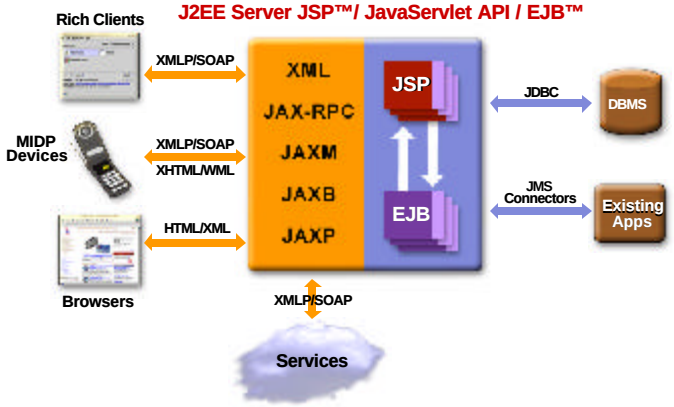
- Best practice guidelines, design patterns and design principles
  - MVC pattern
- Covers all tiers
  - Client tier
  - Web tier
  - Business logic (EJB) tier
  - Database access tier
- Sample code
  - Java Pet Store

Why J2EE for Web Services?

Why J2EE for Web Services?

- Web services is just one of many service delivery channels of J2EE
  - No architectural change is required
  - Existing J2EE components can be easily exposed as Web services
- Many benefits of J2EE are preserved for Web services
  - Portability, Scalability, Reliability
  - No single-vendor lock-in

Web Services Model Over J2EE



Where Are We Now?

- Java APIs for Web Services are being developed very rapidly
  - Web services support on WUST (WSDL, UDDI, SOAP) ready now
  - Next layer Web services work in progress
- Tools are available now for exposing existing J2EE components as Web services
- J2EE community is in the process of defining overall framework for Web Services (J2EE 1.4, Web services for J2EE)

Design Goals J2EE 1.4 Web Services Framework

- Portability of Web services component
  - Over different vendor platform
  - Over different operational environment
- Leveraging existing J2EE programming models for service implementation
- Easy to program and deploy
  - High-level Java APIs
  - Use existing deployment model

J2EE 1.4 Web Services Framework

- J2EE 1.4 (JSR 151)
- Web services for J2EE (JSR 109)
- JAX-RPC (JSR 101)
- JAXR (Java API for XML Registries)
- SAAJ (SOAP with Attachments API for Java)
- EJB 2.1

How to Get Started

Step1: How to Get Started (for Beginners) for Web-tier

- Download and run Java Web Services Developer Pack (Java WSDP) and its tutorial for web-tier programming (Servlet and JSP)
- Java Web Services Developer Pack Download
  - java.sun.com/webservices/downloads/webservicespack.html
- Java Web Services Developer Pack Tutorial
  - java.sun.com/webservices/downloads/webservicestutorial.ht

Step2: How to Get Started (for Beginners) for EJB tier and Others

- Download and run J2EE 1.3.1 Reference Implementation (RI)
  - In the future course, we will use Sun ONE App Server 7 Platform Edition, which is production quality and is also freely downloadable
- Study J2EE Tutorial from java.sun.com
  - developer.java.sun.com/developer/onlineTraining/J2EE/In

Step3: Next Step (For Intermediate J2EE Programmers)

- Leverage J2EE Blueprint
- Download and read J2EE Blueprint to learn the best practice guidelines and design patterns
- Download and run Java Pet Store as an example of J2EE application based on best practice guidelines
  - java.sun.com/blueprints/guidelines/designing_ent

Step4: Next Step (For Advanced J2EE Programmers)

- Try J2EE IDE of your choice. Sun ONE Studio 4 EE has 60 day free license with bunch of sample programs you can try
- Sun ONE Studio 4 EE
  - wwws.sun.com/software/sundev/jde/buy/index.html
- Sun ONE Studio 4 EE tutorial
  - wwws.sun.com/software/sundev/jde/examples/index.html
- Other vendors also have good quality J2EE IDE's
  - BEA, IBM, Borland

Step5: Next Step (For Advanced J2EE Programmers)

- There is no shortage of quality J2EE online resources
  - java.sun.com/j2ee
  - www.theserverside.com

Step6: Next Step (For J2EE 1.4 Features)

- We will use J2EE 1.4 Beta Reference Implementation for building J2EE applications that use 1.4 features (or we might use individually released packages)
  - JAXP, JAX-RPC, JAXR, SAAJ
  - JMX
  - Management and Deployment
  - JACC
  - ...

Next Step (For J2EE 1.4 Features)

- J2EE Application Verification Kit
- EcPerf (SPECJAppServer)

Summary & Resources

Summary

- J2EE is the platform of choice for development and deployment of n-tier, web-based, transactional, component-based enterprise applications
- J2EE is standard-based architecture
- J2EE is all about community
- J2EE evolves according to the needs of the industry

Resources

- J2EE Home page
  - java.sun.com/j2ee
- J2EE Reference Implementation
  - http://java.sun.com/downloads/
- J2EE Tutorial
  - developer.java.sun.com/developer/onlineTraining/J2EE/Intro2/j2ee.html
- J2EE Blueprint
  - java.sun.com/blueprints/guidelines/designing_enterpr

Resources

- Java Web Services Developer Pack Download
  - java.sun.com/webservices/downloads/webservicespack.html
- Java Web Services Developer Pack Tutorial
  - java.sun.com/webservices/downloads/webservicestutorial.html
- Sun ONE Studio 4 EE
  - wwws.sun.com/software/sundev/jde/buy/index.html
- Sun ONE Studio 4 EE tutorial
  - wwws.sun.com/software/sundev/jde/examples/index.html

Questions?